Lab Assignment 10.3 : Code Review and Quality

NAME: G . SRI SAHITH

HT.NO: 2303A52243

Course: AI Assisted Coding (23CS002PC304)

Assignment: 10.3

Theme: Using AI to improve code quality and readability

Problem Statement 1: AI-Assisted Bug Detection

Exact Task Description

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):
    result = 1
    for i in range(1, n):
        result = result * i
    return result
```

Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
    - Identify the logical bug in the code.
    - Explain why the bug occurs (e.g., off-by-one error).
    - Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Prompt

"I have a Python function to calculate factorials, but factorial(5) is giving the wrong answer. Identify the logical bug, explain why it's happening, provide the corrected version, and handle edge cases like zero and negative numbers."

Code

Corrected Code:

```
def factorial(n):
    # Edge case: Negative numbers
    if n < 0:
        return "Error: Factorial does not exist for negative numbers."
    # Edge case: Zero
    if n == 0:
        return 1

    result = 1
    # Bug fixed: changed 'n' to 'n + 1' to include the final number
    for i in range(1, n + 1):
        result *= i
    return result

# --- Test Execution ---
print(f"Factorial of 5: {factorial(5)}")
print(f"Factorial of 0: {factorial(0)}")
print(f"Factorial of -3: {factorial(-3)}")
```

Output

```
Factorial of 5: 120
Factorial of 0: 1
Factorial of -3: Error: Factorial does not exist for negative numbers.
```

Explanation

The original code had a classic "off-by-one" error. In Python, the range(start, stop) function stops *before* the stop number. So, range(1, 5) only gives 1, 2, 3, and 4. It completely skips multiplying by 5, giving 24 instead of 120. The AI easily spotted this and changed it to range(1, n + 1).

Comparison note: When I initially looked at it, I only fixed the loop. However, the AI correctly reminded me about the edge cases. Mathematically, 0! is 1, and factorials for negative numbers don't exist. Adding those checks made the function truly robust.

Problem Statement 2: Improving Readability & Documentation

Exact Task Description

Scenario: The following code works but is poorly written:

```
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
```

```
    # Missing implementation
```

Instructions: Use AI to critique the function's readability, parameter naming, and lack of documentation. Rewrite the function with descriptive names, a complete docstring, exception handling for division by zero, and input validation. Compare versions and test both.

Prompt

"Review this poorly written calculator function. Critique its readability and naming conventions. Then, rewrite it to be production-ready. Give it descriptive variable names, a full docstring, input validation (ensure numbers are passed), and handle division by zero errors gracefully."

Code

Corrected Code:

```
from typing import Union

def calculate_operation(num1: Union[int, float], num2: Union[int, float], operation: str) -> Union[int, float, str]:
    """
    Performs basic arithmetic operations on two numbers.

    Args:
        num1 (int/float): The first number.
        num2 (int/float): The second number.
        operation (str): The operation to perform ('add', 'sub', 'mul', 'div').

    Returns:
        The result of the arithmetic operation, or an error message if invalid.
    """
    # Input validation
    if not isinstance(num1, (int, float)) or not isinstance(num2, (int, float)):
        return "Error: Both inputs must be numeric."

    operation = operation.lower()

    if operation == "add":
        return num1 + num2
    elif operation == "sub":
        return num1 - num2
    elif operation == "mul":
        return num1 * num2
    elif operation == "div":
        try:
            return num1 / num2
        except ZeroDivisionError:
            return "Error: Cannot divide by zero."
    else:
```

```
    return f"Error: Unknown operation '{operation}'."

# --- Test Execution ---
print(f"Addition: {calculate_operation(10, 5, 'add')}")
print(f"Division: {calculate_operation(10, 0, 'div')}")
print(f"Invalid Type: {calculate_operation(10, 'five', 'add')}")
```

Output

Addition: 15
Division: Error: Cannot divide by zero.
Invalid Type: Error: Both inputs must be numeric.

Explanation

The original code was terrible to read. Variables a, b, and c tell us nothing about what they do. Are they apples, bananas, and carrots? The AI refactored a and b to num1 and num2, and c to operation. It also added a docstring, which acts as an instruction manual for the function. The most important fix was for the div branch. If a user passed 0 as the second number, the whole app would crash. Wrapping it in a try-except block catches that specific math error and returns a polite message instead.

Problem Statement 3: Enforcing Coding Standards

Exact Task Description

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):
  for i in range(2, n):
    if n % i == 0:
      return False
  return True
```

Instructions: Verify the function works correctly. Use an AI tool to list all PEP8 violations and refactor the code (function name, spacing, indentation, naming). Write a short note on how automated AI reviews could streamline code reviews in large teams.

Prompt

"Act as a strict code reviewer. Identify all PEP8 violations in this prime-checking function and provide a refactored, perfectly formatted version."

Code

Corrected Code:

```
def check_prime(number: int) -> bool:
  """Check if a given integer is a prime number."""
  # Edge cases for numbers less than 2
  if number < 2:
```

```
        return False

    for i in range(2, number):
        if number % i == 0:
            return False

    return True

# --- Test Execution ---
print(f"Is 7 prime? {check_prime(7)}")
print(f"Is 10 prime? {check_prime(10)}")
print(f"Is 1 prime? {check_prime(1)}")
```

Output

```
Is 7 prime? True
Is 10 prime? False
Is 1 prime? False
```

Explanation

PEP8 is the official style guide for Python. The AI immediately flagged the function name Checkprime. In Python, function names must be lowercase with underscores (check_prime), whereas capital letters are reserved for Class names. It also added type hints and a docstring.

Reflection on AI in Teams: In a large software team, developers spend hours arguing over spacing, naming, and formatting in pull requests. Using an AI linter automatically fixes these cosmetic issues instantly. This streamlines the review process, allowing senior engineers to focus purely on the core logic and architecture rather than formatting rules.

Problem Statement 4: AI as a Code Reviewer in Real Projects

Exact Task Description

Scenario: In a GitHub project, a teammate submits:

```
def processData(d):
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions: Manually review the function for readability, modularity, and edge cases. Use AI to generate a code review covering better naming, input validation, type hints, and generalization. Refactor based on AI feedback and write a short reflection on whether AI should be a standalone reviewer or an assistant.

Prompt

"Review this Python function processData(d). Suggest improvements for better naming, intent clarity, type hints, and input validation (handling non-list inputs or non-integers). Refactor it to be more generalized."

Code

Corrected Code:

```
from typing import List, Union

def double_even_numbers(numbers: List[Union[int, float]]) -> List[Union[int, float]]:
    """
    Filters a list for even numbers and returns them doubled.
    """
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list.")

    processed_list = []
    for num in numbers:
        # Validate that the element is a number before applying modulo
        if isinstance(num, (int, float)) and num % 2 == 0:
            processed_list.append(num * 2)

    return processed_list

# --- Test Execution ---
data = [1, 2, "three", 4, 5.5, 6]
print(f"Processed: {double_even_numbers(data)}")
```

Output

```
Processed: [4, 8, 12]
```

Explanation

The original one-liner was clever but highly fragile. processData(d) gives zero context about what the function does. If you pass a list with a string inside it (like [2, 'a']), the x % 2 check causes a fatal crash. The AI suggested renaming it to double_even_numbers and adding strict type checks isinstance(num, (int, float)) inside the loop so it ignores invalid data gracefully.

Reflection: AI should be an **assistant**, not a standalone reviewer. While AI is brilliant at catching edge cases (like the string crash) and enforcing syntax, it lacks business context. An AI doesn't know *why* we are doubling these numbers or if doubling them aligns with the company's product goals. It should act as a "first pass" filter before a human reviews the actual business logic.

Problem Statement 5: AI-Assisted Performance Optimization

Exact Task Description

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
```

```
    return total
```

Instructions: Test the function with a large list. Use AI to analyze time complexity, suggest performance improvements (e.g., built-in functions), and provide an optimized version. Compare execution times and discuss trade-offs.

Prompt

"Analyze the time complexity of this sum_of_squares function. Suggest a more performant, optimized version using Python's built-in capabilities or generators. Include code to measure and compare the execution time of both versions."

Code

Corrected Code:

```
import time

# Original Function
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total

# AI-Optimized Function using Generator Expression
def sum_of_squares_optimized(numbers):
    return sum(x * x for x in numbers)

# --- Test Execution & Time Comparison ---
large_dataset = list(range(1000000))

# Test Original
start_time = time.time()
sum_of_squares(large_dataset)
original_duration = time.time() - start_time

# Test Optimized
start_time = time.time()
sum_of_squares_optimized(large_dataset)
optimized_duration = time.time() - start_time

print(f"Original Time:  {original_duration:.4f} seconds")
print(f"Optimized Time: {optimized_duration:.4f} seconds")
```

Output

Original Time: 0.0821 seconds
Optimized Time: 0.0514 seconds

Explanation

Both functions have a Time Complexity of **O(N)**, meaning they have to look at every single number in the list once. However, the *optimized* version is faster in reality. The original code uses a manual for loop, which executes at the Python level and carries overhead for variable assignment (total +=). The optimized version uses a generator expression wrapped in Python's built-in sum() function. The built-in sum() is written in highly optimized C code under the hood.

Trade-offs: Usually, squeezing out performance makes code harder to read. However, Python is unique here. The optimized version sum(x * x for x in numbers) reads exactly like mathematical English ("the sum of x times x for every x in the numbers"). In this case,